

Instruction to candidates:

Your program code and output for each of Task 1 to 4.3 should be saved in a single .ipynb file. For example, your program code and output for Task 1 should be saved as:

TASK1_<your name>_<centre number>_<index number>.ipynb

1 Name your Jupyter Notebook as:

Task1_<your name>_<center number>_<index number>.ipynb

A music app wants to predict the genre of new songs using their lengths and tempos.

For each of the sub-tasks, add a comment statement at the beginning of the code using the hash symbol '#' to indicate the sub-task the program code belongs to, for example:

In [1]:

```
#Task 1.1  
Program Code
```

Output:

Task 1.1

A list of sample songs is saved in the file `song.txt`, in ascending order of the length, and then ascending order of the tempo. Write a function `read_file(filename)` that

- reads a text file `filename`
- stores the data in a list with 3 columns length, tempo and genre
- stores length and tempo as integers, and genre as a string
- returns the list

Use the following code to test your function.

```
print(read_file('song.txt'))
```

[4]

Given a new song with length and tempo provided, we can search for sample songs with the same length.

Task 1.2

Write a function `binary_search(song_list, new_length)` that takes two parameters and searches for sample songs with `new_length` in `song_list` using the binary search algorithm. The function returns the list of possible genres of the sample songs found, or an empty list if no sample songs are found.

Use the following code to test your function.

```
print(binary_search(read_file('song.txt'), 240) == [])
print(binary_search(read_file('song.txt'), 260) == ['Rock', 'Rock'])
```

[7]

If we cannot find any sample songs with the same length as the new song, we can find the five nearest neighbours of the new song. The distance between song A and song B are calculated using their lengths and tempos as follows:

$$\text{Distance} = \sqrt{(\text{song_A_length} - \text{song_B_length})^2 + (\text{song_A_tempo} - \text{song_B_tempo})^2}$$

Task 1.3

Write a function `distance(song_list, new_length, new_tempo)` that

- calculates the distance of each sample song in `song_list` to the new song with `new_length` and `new_tempo`
- inserts all the distances into `song_list` as the fourth column
- returns `song_list`

Use the following code to test your function.

```
print(distance(read_file('song.txt'), 280, 115))
```

[3]

Task 1.4

Write a program to sort `song_list` obtained from Task 1.3 in ascending order of the distance from the new song, using the quick sort algorithm. Output the genre of the five nearest neighbours of the new song.

[6]

Given a new sample song with length, tempo and genre provided, we can update the list of sample songs using insertion sort.

Task 1.5

Write a program that

- inserts the new sample song into the sorted list obtained from Task 1.1 using the insertion sort algorithm
- ensures that the sample songs are listed in ascending order of the length, and then ascending order of the tempo
- writes the updated list into another file `new_song.txt`

Test your program with a new sample song of length 310, tempo 120 and genre 'Classical'. [6]

Save your Jupyter Notebook for Task 1.

2 Name your Jupyter Notebook as:

Task2_<your name>_<center number>_<index number>.ipynb

A music streaming application allows users to create playlists. Each playlist stores songs in the order in which they will be played. Each song has a unique title and an associated artist.

The song at the front of the playlist is always the next song to be played. When a song is played or skipped, it is removed from the playlist, allowing the next song to become the first song in the playlist. The playlist is implemented using a linked list.

For each of the sub-tasks, add a comment statement at the beginning of the code using the hash symbol '#' to indicate the sub-task the program code belongs to, for example:

In [1]:

```
#Task 2.1
Program Code
```

Output:

Task 2.1

The class SongNode contains three attributes:

- title
- artist
- next, a reference to the next SongNode in the playlist

Write program code for the SongNode class.

[2]

Task 2.2

The class `Playlist` contains one attribute:

- `head`: a reference to the first `SongNode` in the playlist

The class `Playlist` contains the following methods:

- a constructor that initialises `head` to appropriate values for an empty playlist
- `isEmpty()` that returns `True` if the playlist is empty, or `False` otherwise
- `addSong()` that takes a `SongNode` as a parameter and adds it to the end of the playlist
- `playNext()` that removes and returns the first song in the playlist. Returns `None` if the playlist is empty
- `displayPlaylist()` that displays the songs in the playlist in order

Write program code for the `Playlist` class.

[9]

Task 2.3

The text file `PLAYLIST.txt` stores song data in the following format:
title,artist

Example:

Star-filled Sky,Warmplay

Write program code to:

- initialise a `Playlist` object named `my_playlist`
- read all songs from `PLAYLIST.txt`
- instantiate each song as a `SongNode`
- call `addSong()` with each new `SongNode`
- display all songs in the playlist
- call `playNext()` twice to play two songs in the playlist
- display the remaining playlist

Test your program and show the output. All outputs should have appropriate messages to indicate what they are showing.

[4]

Task 2.4

Amend the `Playlist` class to include the following methods:

- `search()` that takes a `title` as a parameter and searches for the song with the given title. It returns the corresponding `SongNode` found, otherwise returns `None`.
- `delete()` that takes a `title` as a parameter and removes the corresponding song from the playlist. It returns `True` if the song is successfully removed, or `False` otherwise
- `insert()` that takes `position` and a `SongNode` as parameters and inserts `SongNode` into the playlist at the specified position. If `position` is 1 or less, `SongNode` is inserted at the beginning of the playlist. If `position` is greater than the number of songs in the playlist, `SongNode` is inserted at the end.

In a new cell, write program code to:

- search for "Silver" and display the result
- delete "Mom's Pasta"
- display the updated playlist
- create a `SongNode` with:
 - title: Smile Till the End
 - artist: Ms Gugu & Uno Cars
- insert the song into position 3
- display the updated playlist

Test your program and show the output. All outputs should have appropriate messages to indicate what they are showing.

[11]

Save your Jupyter Notebook for Task 2.

3 Name your Jupyter Notebook as:

Task3_<your name>_<center number>_<index number>.ipynb

For each of the sub-tasks, add a comment statement at the beginning of the code using the hash symbol '#', to indicate the sub-task the program code belongs to, for example:

In [1]:

```
#Task 3.1
Program Code
```

Output:

The Vigenère Cipher uses a 26×26 table containing the letters A to Z as both the row and column headings. This table is known as the Vigenère Table. The first row of the table contains the 26 letters of the English alphabet in their usual order:

ABCDEFGHIJKLMNOPQRSTUVWXYZ

Beginning from the second row, each subsequent row is formed by shifting the letters one position to the left in a cyclic manner. For example, in the second row, the letter B becomes the first letter, while A moves to the end of the row:

BCDEFGHIJKLMNOPQRSTUVWXYZA

This shifting pattern continues until all 26 rows are formed.

An example of the Vigenère Table is as shown:

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
...																									
X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y

Task 3.1

Write a function `task3_1()` that creates and returns a Vigenère Table as a 2-dimensional list. Call the function `task3_1()` and then output the contents of the Vigenère Table. [8]

Suppose we want to encrypt the plaintext: HELLO WORLD WRITTEN IN JAVA using the keyword: PYTHON using Vigenère Cipher. The steps to encrypt the plaintext are as follows:

- Remove all spaces and punctuation from the plaintext and convert all characters to upper case.
- Write the keyword, character by character, under the plaintext. Repeat the keyword until it matches the length of the plaintext as shown below.

```
HELLOWORLDWRITTENINJAVA
PYTHONPYTHONPYTHONPYTHO
```

- Label the Vigenère table with rows A-Z and columns A-Z. Pick a letter in the plaintext and its corresponding letter in the keyword, use the keyword and plaintext letter as the row index and column index, respectively, and the entry at the row-column intersection is the letter in the cipher text. For example, L in the plaintext and H in the keyword gives S in the cipher text.

```
HELLOWORLDWRITTENINJAVA
PYTHONPYTHONPYTHONPYTHO
WCESCJDPEKKEXRMLBVCHTCO
```

- Divide the cipher text into 5-letter blocks

```
WCESC JDPEK KEXRM LBVCH TCO
```

Task 3.2

Write a function `task3_2(keyword, plaintext, vtable)` that takes in three parameters. The function uses Vigenère Cipher to encode the plaintext using the keyword and Vigenère Table `vtable`, and returns the 5-letter blocks cipher text. [10]

Task 3.3

Use `task3_1()` and `task3_2()` to encode the message "Computing is an interesting subject!" with keyword "PYTHON" and print the cipher message. [2]

Save your Jupyter Notebook for Task 3.

4 Name your Jupyter Notebook as:

Task4_<your name>_<center number>_<index number>.ipynb

A clinic wants to create a suitable database to store patients' data and current queue information. The database will have two tables: a table to store data about patients and a table about queue. The fields in each table are:

Patient:

- PatientID - unique patient number, for example, 2
- Name - the name of patient
- Age - the age of patient
- Gender - the gender of patient

Queue:

- ID - unique number that is assigned based on the sequence the record is inserted into the table
- PatientID - patient number, for example, 2
- Priority - the priority of the queue, for example, 1 which is the highest priority
- InQueue - the status whether patient is in the queue, for example, 1 is in queue and 0 is not in queue.

For each of the sub-tasks, add a comment statement at the beginning of the code using the hash symbol '#', to indicate the sub-task the program code belongs to, for example:

In [1]:

```
#Task 4.1
Program Code
```

Output:

Task 4.1

Write a Python program that uses SQL code to create the database CLINIC with the two tables given. Define the primary and foreign keys for each table. [5]

Task 4.2

The text files patient.txt and queue.txt store the data for each of the tables in the database.

The data in `patient.txt` is stored in the following format:
 patient ID, name, age, gender

The data in `queue.txt` is stored in the following format:
 patient ID, priority, inqueue

Write a python program to read in the data from each file and store each item of data in the correct place in the database. [4]

Task 4.3

Write a Python program to input a patient name and display the name, age and gender of the patient if the name of the patient is in the database, otherwise display "Not found". Test your program with the name John and Mary. [5]

Save your Jupyter Notebook.

Task 4.4

Write a Python program and the necessary files to create a web application that enables the list of patients in the queue to be displayed.

For each record the web page should include the:

- name
- age
- gender

The program should return an HTML document that enables the web browser to display a table with the required data sorted by the priority in ascending order followed by the sequence in the queue.

Save your Python program as:

`Task4_4_<your name>_<center number>_<index number>.py`

With any additional files/ subfolders in a folder named:

`Task4_4_<your name>_<center number>_<index number>` [9]

Run the web application and save the output of the program as

`Task4_4_<your name>_<center number>_<index number>.html` [1]